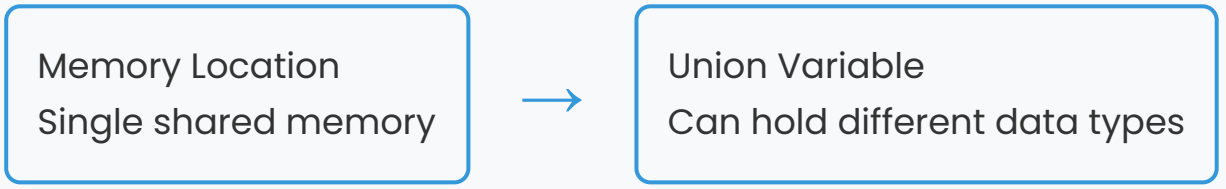
Unions in C Programming

Understanding unions for efficient memory usage in C programming

Introduction to Unions

A union is a special data type available in C that allows storing different data types in the same memory location. You can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple purposes.

Example: In case of support price of shares you require only the latest quotations. And only the ones that have changed need to be stored. So if we declare a structure for all the scripts, it will only lead to crowding of the memory space. Hence it is beneficial if we allocate space to only one of the members. This is achieved with the concepts of UNIONS.



Union Declaration

A union is declared and used in the same way as a structure, but with the keyword `union` instead of `struct`. The syntax is:

Union Declaration Syntax

```
union union-tag
{
  datatype variable1;
  datatype variable2;
  datatype variable3;
  ……….
};
```

The union tag is optional and each member definition is a normal variable definition, such as int i; or float f; or any other valid variable definition. At the end of the union's definition, before the

final semicolon, you can specify one or more union variables but it is optional.

Union vs Structure

UNIONS are similar to STRUCTURES in all respects but differ in the concept of storage space. A UNION stores all its members in the same area. This means that only one member of a union can be used at a time. This is the main difference between a union and a structure.

Structure		Union
Separate memory for each member	↔	Shared memory for all members

Unions provide an efficient way of using the same memory location for multiple purposes.

Initializing a Union

Initializing a union in programming involves declaring a union variable and assigning initial values to its members. A union is a data structure that allows storing different data types in the same memory location.

Union Initialization Example

```
union temp
{
  int x;
  char y;
```

```
  float z;
};
```

Data Assignment

```
temp.x = 10;
temp.y = 20;
temp.z = 30;
```

When initializing a union variable, you can assign values to individual members or initialize the entire union at once.

Accessing Members of a Union

To access any member of a union, we use the member access operator (.). The member access operator is coded as a period between the union variable name and the union member that we wish to access.

Accessing Union Members

```
union data;
data.i = 10;
data.f = 220.5;
strcpy( data.str, "C Programming");
printf( "data.i : %d\n", data.i);
printf( "data.f : %f\n", data.f);
printf( "data.str : %s\n", data.str);
return 0;
}
```

You would use the keyword union to define variables of union type.

The following example to use unions in a program:

Complete Union Example

```c
#include<stdio.h>
#include<string.h>
union Data
{
  int i;
  float f;
  char str[20];
};
int main( )
{
  union Data data;
  data.i = 10;
  data.f = 220.5;
  strcpy( data.str, "C Programming");
printf( "data.i : %d\n", data.i);
printf( "data.f : %f\n", data.f);
printf( "data.str : %s\n", data.str);
return 0;
}
```

Output of Union Example

Program Output

```
data.i : 1917853763
data.f : 41223605803277948604527599943 68.000000
data.str : C Programming
```

Explanation

Data Storage

The union stores different data types in the same memory location

Value Assignment

Only one member can be active at a time

Display

printf() displays the current active member value

Here, values of i and f members of union got corrupted because the final value assigned to variable has occupied the memory location and this is the reason that the value of str member is getting printed very well.

Date Example with Unions

In case of support price of shares you require only the latest quotations. And only the ones that have changed need to be stored. So if we declare a structure for all the scripts, it will only lead to crowding of the memory space. Hence it is beneficial if we allocate space to only one of the members. This is achieved with the concepts of UNIONS.

Date Structure Definition

```
union date_tag
{
  char complete_date[9];
  char month[2];
  char break_value1;
  char day[2];
  char year[2];
};
struct part_date
{
  char month[2];
  char break_value2;
  char day[2];
  char year[2];
};
parrt_date;
}
```

Date Assignment

```
date = {"01/05"};
parrt_date.month = "Jan";
parrt_date.day = 1;
parrt_date.break_value1 = "Sun";
parrt_date.year = 2023;
```

Now, use one variable at a time which is the main purpose of having unions. This is achieved with the concepts of UNIONS.

Conclusion

Key Takeaways

Memory Efficiency

Unions allow multiple data types in the same memory location

Flexible Design

Only one member can be active at a time

Member Access

Accessed using dot (.) operator

Application Areas

Useful for date handling, variant data types, and memory optimization

Unions vs Structures

Memory Usage

Structures allocate separate memory for each member

Shared Memory

Unions share memory among all members

Unions are particularly useful when you need to store different data types in the same memory location but only use one at a time. They are commonly used in embedded systems and for

creating variant data structures.

Best Practices

Initialization

Initialize all members or use dot notation to access specific members

Type Safety

Be aware of which member is active when accessing

Unions are fundamental to advanced C programming and complement structures for creating flexible and memory-efficient data structures.